"It's about events, not calls."

MODULE 30

Event-Driven Architecture and Apache Kafka

Design scalable, real-time systems using event-driven principles and Apache Kafka to build resilient, decoupled enterprise architectures.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 30 OVERVIEW

Design scalable, real-time systems using event-driven principles and Apache Kafka to build resilient, decoupled architectures.

Kafka is the backbone of modern event-driven enterprises -- this module covers the concepts, the platform internals, and a hands-on lab building real Kafka topics, producers, and consumers.

DURATION & LAB

1.5 Hours Theory

Event-driven architecture, Kafka fundamentals, brokers, topics, partitions, replication, and consumer groups.

2 Hours Lab

Northstar CRM Topics: a local Kafka broker, keyed CRM events, Java producers, and consumer groups.

Scenario

E-commerce order processing: order, payment, inventory, and notification services reacting to Kafka events.

MODULE ARC

Understand EDA

Why event-driven beats request-response for scale, resilience, and flexibility.

Master Kafka Internals

Brokers, topics, partitions, replication, producers, consumers, and consumer groups.

Build the Lab

Stand up a real Kafka broker and publish/consume keyed CRM events end to end.

KEY TAKEAWAY Total time: 3.5 hours (1.5 hours theory + 2 hours hands-on lab). By the end, you will design and reason about event-driven systems using Apache Kafka.

WHY EVENT-DRIVEN SYSTEMS MATTER

Modern enterprises need systems that are fast, scalable, resilient, and adaptive. Event-driven architecture helps achieve exactly that.

WHAT EVENT-DRIVEN ARCHITECTURE DELIVERS

- **Real-Time Responsiveness —** events are processed as they happen, enabling instantaneous detection, alerts, and actions.
- **Scalability —** event-driven systems scale horizontally by adding more consumers and processing events in parallel.
- **Decoupling & Flexibility —** producers and consumers are independent, letting teams evolve services without impacting others.
- **Resilience & Fault Tolerance —** events can be retried, replayed, and processed even if components are temporarily unavailable.
- **Better Business Outcomes —** faster insights, improved customer experience, and the ability to respond to business changes quickly.

BUSINESS IMPACT

- **Lower Latency —** process events in real time for faster decisions.
- **Higher Throughput —** handle millions of events per second efficiently.
- **Cost Efficiency —** scale only the components that need to scale.
- **Innovation Enablement —** easily integrate new services and data sources.

REAL-WORLD EXAMPLE

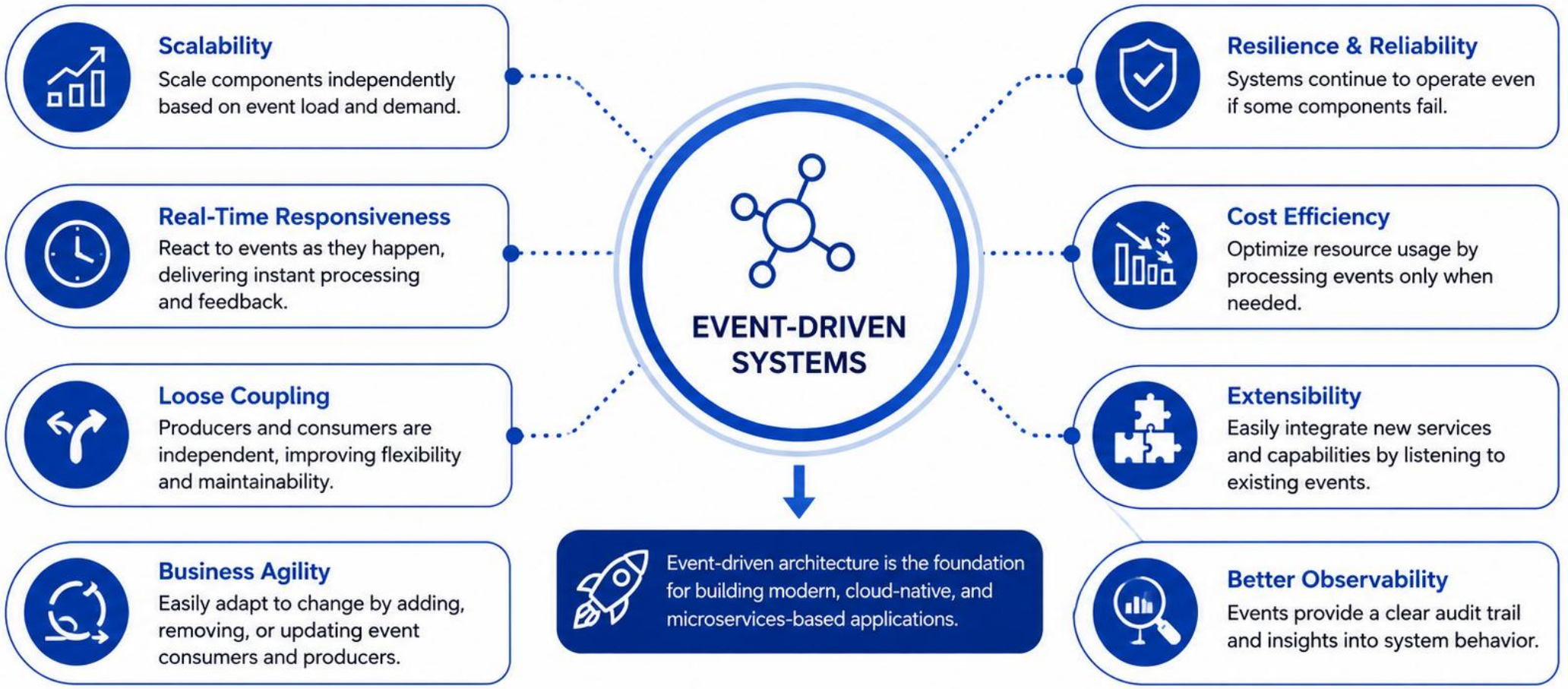
Global companies like Netflix, Uber, LinkedIn, and Amazon use event-driven architecture with Kafka to power real-time recommendations, tracking, fraud detection, and more.

Netflix **Uber** **LinkedIn** **Amazon**

KEY TAKEAWAY Event-driven systems are no longer a choice -- they are a necessity. They help organizations build systems that are agile, future-ready, and customer-centric.

Why Event-Driven Systems Matter

Event-driven systems enable modern applications to be **scalable, responsive, and resilient.**



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design, implement, and scale event-driven solutions using Apache Kafka.

- **1. Understand Event-Driven Architecture —** explain the principles, benefits, and core components of event-driven systems.
- **2. Explore Kafka Fundamentals —** understand Kafka architecture, brokers, topics, partitions, replication, and messaging concepts.
- **3. Work with Producers and Consumers —** develop Kafka producers and consumers and understand consumer groups, message ordering, and retention.
- **4. Analyze Event Processing Flow —** trace end-to-end event flow from production to consumption and understand processing patterns.
- **5. Design Decoupled Enterprise Systems —** apply events to decouple services and build scalable, resilient, maintainable enterprise architectures.
- **6. Apply Kafka in Real-World Scenarios —** identify enterprise use cases and implement event-driven solutions using Kafka in practical applications.

KEY TAKEAWAY Outcome: you will be able to design, implement, and scale event-driven solutions using Apache Kafka to enable real-time, reliable, decoupled systems for enterprise applications.

ENTERPRISE SCENARIO: E-COMMERCE ORDER PROCESSING

An online store receives thousands of orders every minute. Event-driven architecture with Kafka enables real-time, scalable, reliable processing.

1. **Order Placed**
2. **Order Created Event**
3. **Inventory Reserved**
4. **Payment Processed**
5. **Order Shipped**
6. **Customer Notified**

BUSINESS CHALLENGES

- High order volume and traffic spikes.
- Multiple services tightly coupled.
- Need for real-time inventory and payment updates.
- Failures in one service impact others.
- Complex scaling and slow time to market.

EVENT-DRIVEN SOLUTION

- Services communicate via events.
- Asynchronous and decoupled processing.
- Real-time updates across the ecosystem.
- Fault isolation and high availability.
- Independent scaling and faster feature delivery.

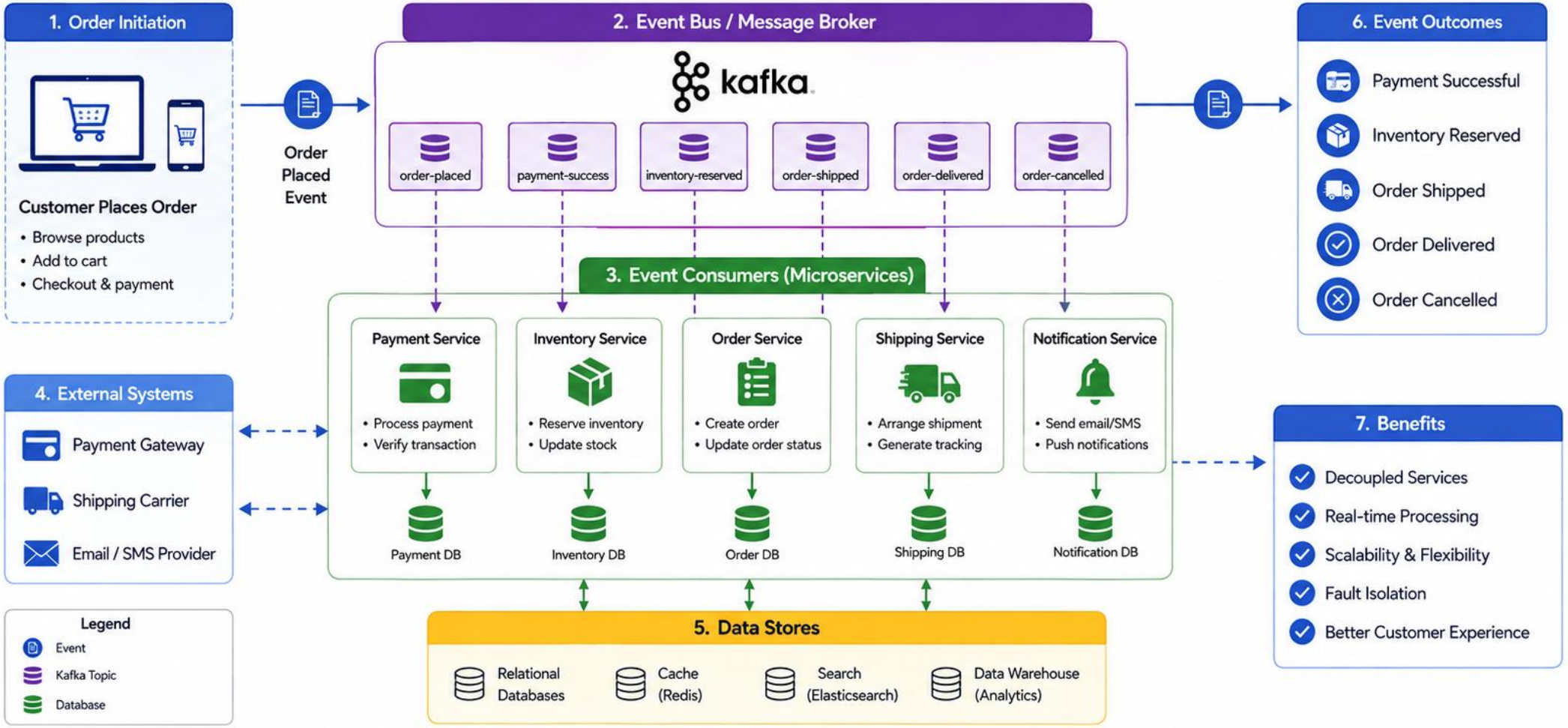
BUSINESS IMPACT

- **Speed —** process orders and updates in real time.
- **Scalability —** handle millions of events seamlessly.
- **Reliability —** ensure delivery even during failures.
- **Flexibility —** add new services without changes.

KEY TAKEAWAY By leveraging Kafka and event-driven architecture, our e-commerce platform can process orders in real time, scale effortlessly, and deliver a seamless customer experience.

Enterprise Scenario: E-Commerce Order Processing

Events flow across decoupled services enabling real-time processing, scalability, and reliability.



WHAT IS EVENT-DRIVEN ARCHITECTURE?

EDA is a software design paradigm where applications communicate by producing, detecting, and consuming events -- significant changes or occurrences in the system.

1 **Event Producer**

Detects a change, publishes an event.

2 **Event Broker (Kafka)**

Receives, stores, and distributes events.

3 **Event Consumers**

Interested services consume and act.

CORE CHARACTERISTICS

- **Asynchronous —** producers and consumers operate independently in time.
- **Decoupled —** services don't need direct knowledge of each other.
- **Scalable —** each component can scale independently.
- **Resilient —** failures in one service don't stop the flow of events.
- **Real-Time Capable —** events enable instant processing and timely insights.

EXAMPLE EVENTS IN A SYSTEM

Order Placed **Payment Completed** **Inventory Updated**

Order Shipped **Email Sent**

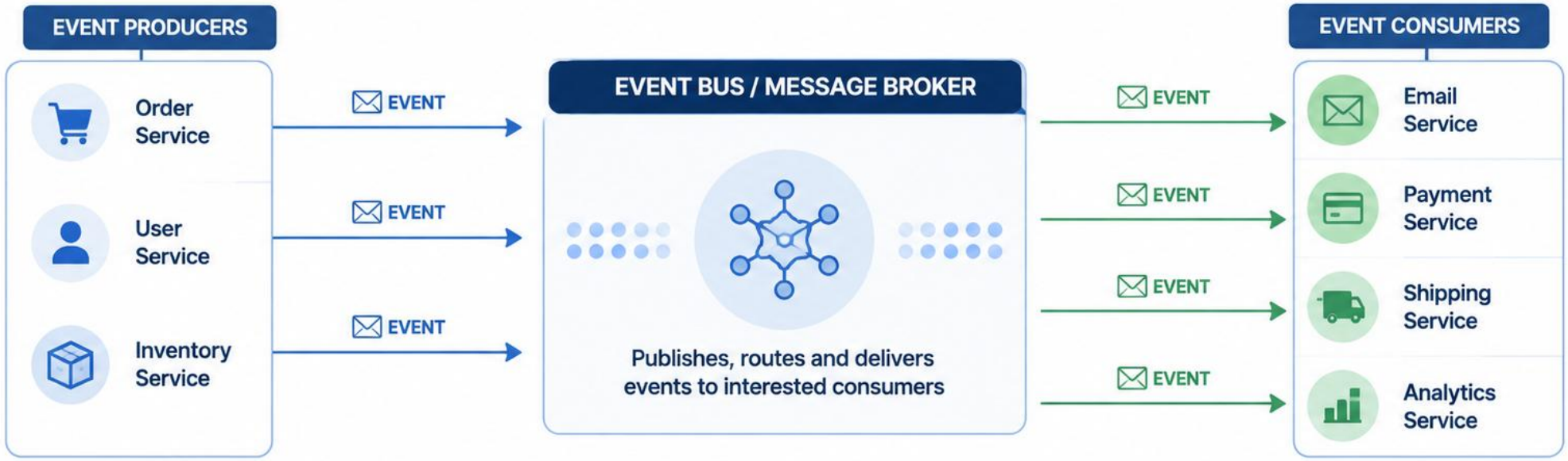
BENEFITS OF EDA

- Improves responsiveness to business changes.
- Enables real-time data processing.
- Supports microservices and distributed systems.
- Enhances reliability, fault tolerance, and integration simplicity.

KEY TAKEAWAY Instead of services calling each other directly (request-response), they publish events when something happens; other services subscribe and react asynchronously. It's about events, not calls.

What is Event-Driven Architecture?

Event-Driven Architecture (EDA) is a design pattern where systems communicate by producing, detecting, consuming, and reacting to **events**.



KEY BENEFITS

LOOSE COUPLING
Producers and consumers are independent.

SCALABILITY
Components can scale independently.

REAL-TIME
Events enable instant response to changes.

RESILIENCE
Systems remain resilient and fault-tolerant.

FLEXIBILITY
Easy to add new consumers or event types.

TRADITIONAL REQUEST-RESPONSE VS EVENT-DRIVEN SYSTEMS

Two fundamental approaches to building distributed systems. Understanding the differences helps you choose the right architecture for the right problem.

Aspect	Traditional Request-Response	Event-Driven Systems
Communication	Synchronous -- caller waits for a response.	Asynchronous -- no waiting.
Coupling	Tight coupling between services.	Loose coupling between services.
Scalability	Limited by the slowest service in the chain.	Highly scalable; each component scales independently.
Resilience	Failures can cascade and affect other services.	Failures are isolated; the system keeps flowing.
Use Cases	Simple workflows, real-time user interactions.	Real-time processing, streaming, integrations, analytics.

HOW EACH WORKS

Request-Response	Event-Driven
A client or service sends a request to another service and waits for a response before continuing.	A producer publishes an event to an event broker. Interested services subscribe and process events asynchronously.

MOST REAL SYSTEMS USE BOTH

REST APIs for synchronous reads/writes

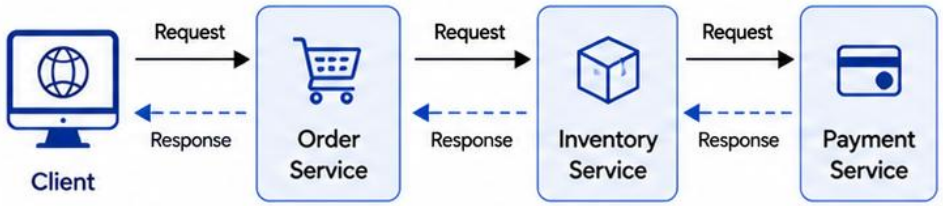
Kafka events for asynchronous fan-out

KEY TAKEAWAY Request-response is great for simple, direct interactions. Event-driven architecture excels in building scalable, resilient, and real-time systems.

Traditional Request-Response vs Event-Driven Systems

Traditional Request-Response

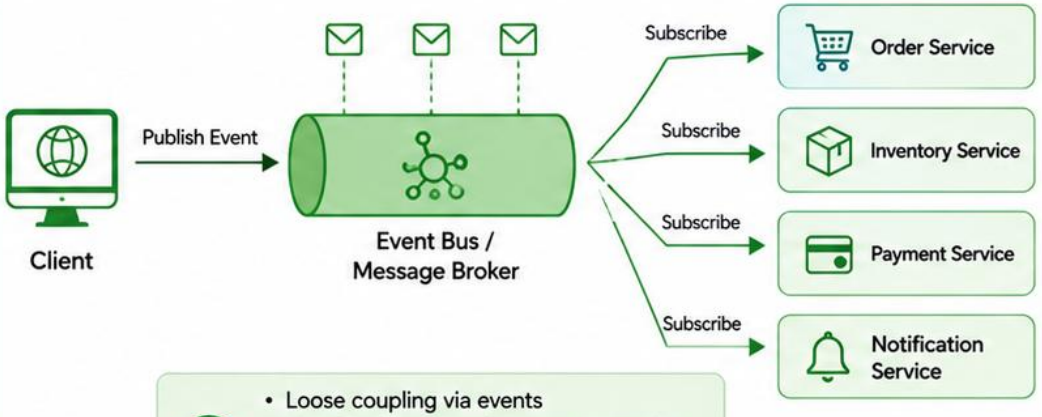
Synchronous communication — tightly coupled



- Tight coupling between services
- Blocking calls and waits
- Failure in one service impacts all
- Limited scalability

Event-Driven Systems

Asynchronous communication — loosely coupled



- Loose coupling via events
- Non-blocking, asynchronous processing
- Failure isolation and resilience
- Highly scalable and flexible

Aspect	Traditional Request-Response	vs	Event-Driven Systems
Coupling	Tight coupling	vs	Loose coupling
Communication	Synchronous (blocking)	vs	Asynchronous (non-blocking)
Failure Handling	Failure propagates through the chain	vs	Failures are isolated
Scalability	Limited by slowest service	vs	Independent scaling of services
Flexibility	Hard to change or extend	vs	Easy to add new services
Performance	Higher latency (waiting for responses)	vs	Lower latency, real-time processing

TRY IT YOURSELF — EXERCISE 1: WHY ASYNC FOR CRM

Reinforces: explaining why Northstar notifications should not block the Customer HTTP API — an analysis exercise.

THE SCENARIO

Customer service creates CUS-1001 Amina Khan over HTTP with correlation lab-request-001. Imagine it also calls email, audit, and analytics synchronously in that same request thread, before it can return a response to the caller.

STEPS (IN notes/lab30-prelab-eda.md)

Step	What To Do
1 — List Sync Pain	Name three concrete problems if the Customer service also calls email, audit, and analytics synchronously inside the same HTTP request thread.
2 — Event Idea	In one sentence, describe publishing a CustomerCreated event so email, audit, and analytics each consume it independently.
3 — Coupling Check	Mark true/false: 'The Customer JVM must be up for the Audit consumer to process an already-published event.' Justify your answer in one line.
4 — Capture Notes	Save your three answers under notes/lab30-prelab-eda.md so they are ready to compare against the reference in class.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-eda-why-async.md (workspace: examples/module-30-exercises).

BENEFITS OF EVENT-DRIVEN DESIGN

Event-driven architecture enables organizations to build systems that are scalable, resilient, and responsive to change -- turning events into real-time insights and actions.

1. Real-Time Responsiveness

Events are captured and processed as they happen, enabling instant detection and faster decisions.

2. Scalability

Handle high event volumes by distributing work across multiple consumers.

3. Loose Coupling

Producers and consumers are independent -- services communicate through events, not direct calls.

4. Flexibility & Agility

Add new services by subscribing to events -- no changes required in existing services.

5. Resilience & Fault Tolerance

Events are persisted and can be retried; failures are isolated and don't bring down the system.

6. Improved Throughput

Asynchronous processing and parallel consumption increase throughput and resource use.

7. Better Data Insights

A rich stream of events feeds analytics, monitoring, auditing, and machine learning.

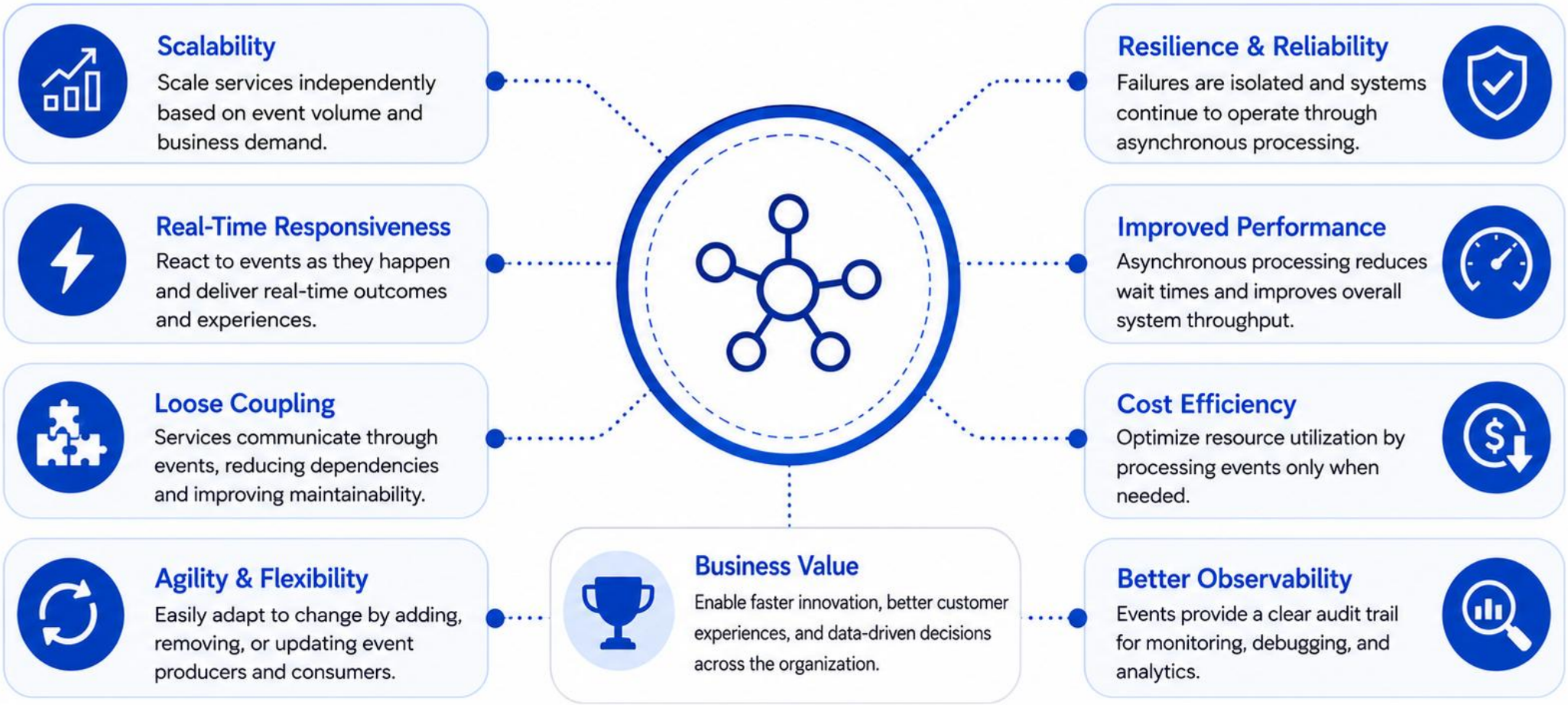
8. Cost Efficiency

Independent scaling and efficient resource usage reduce operational costs over time.

KEY TAKEAWAY Event-driven design empowers organizations to build systems that are fast, scalable, resilient, and adaptable -- driving innovation and better customer experiences.

Benefits of Event-Driven Design

Event-driven design enables systems to be agile, resilient, and built for scale.



CORE COMPONENTS OF EVENT-DRIVEN SYSTEMS

Event-driven systems are built from a set of core components that work together to produce, deliver, and consume events reliably and at scale.

1 **1. Producers**
Generate events.

2 **2. Event Broker**
Stores & routes (e.g. Kafka).

3 **3. Consumers**
Process events.

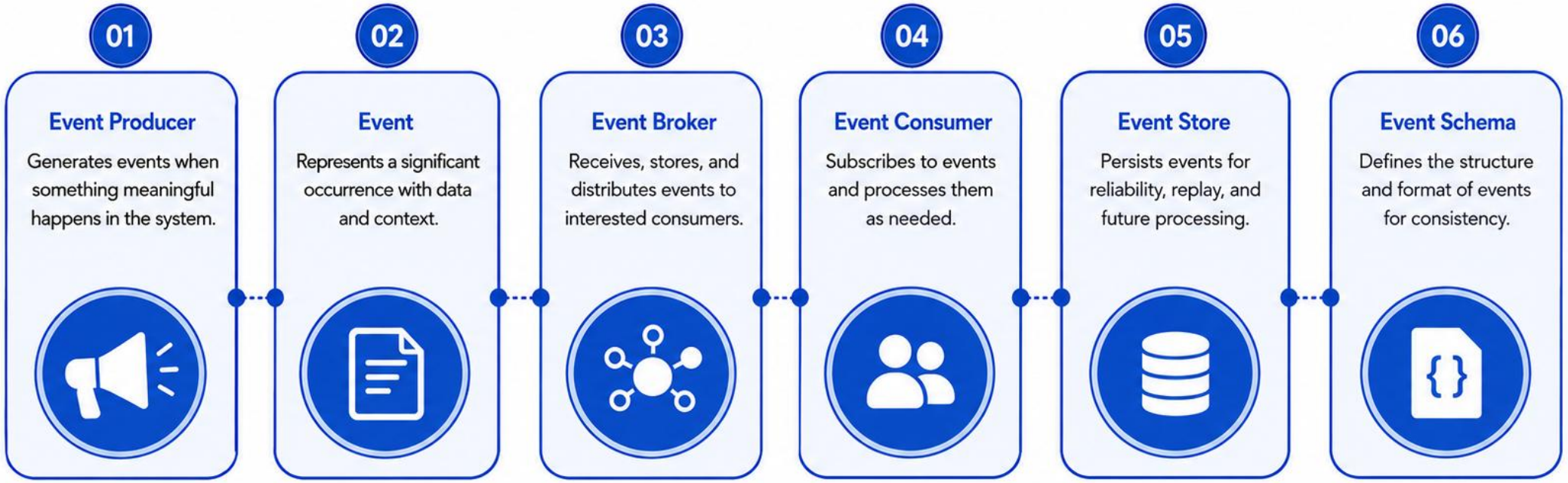
4 **4. Data Stores**
Persist data (optional).

5 **5. External Systems**
Receive alerts/updates.

Component	Role	Examples
Producers	Create and publish events; focus on event creation, not on who consumes it.	Order Service, Payment Service, IoT Device
Event Broker	Central hub for event ingestion and distribution; ensures durability, ordering, and replay.	Apache Kafka, RabbitMQ, Amazon EventBridge
Consumers	Subscribe to events from the broker; process independently and at their own pace.	Notification Service, Analytics Service
Data Stores	Store processed data for operational or analytical use (optional).	PostgreSQL, MongoDB, Redis, S3
External Systems	Receive outputs such as notifications, dashboards, or API calls.	Email/SMS Gateways, Webhooks, Monitoring Tools

KEY TAKEAWAY These core components enable systems to be decoupled, scalable, resilient, and real-time -- forming the foundation of modern event-driven architectures.

Core Components of Event-Driven Systems



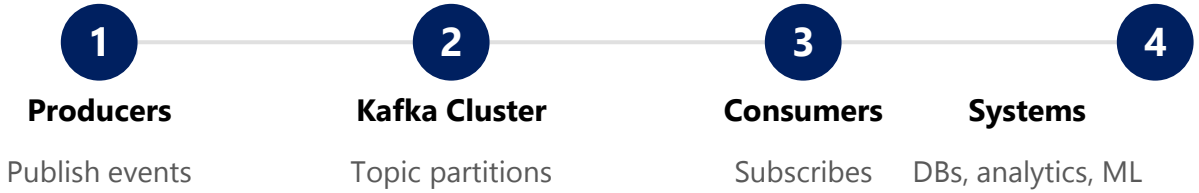
INTRODUCTION TO APACHE KAFKA

Apache Kafka is a distributed event streaming platform designed for high-throughput, fault-tolerant, real-time data pipelines and streaming applications.

WHAT IS APACHE KAFKA?

- Kafka is an open-source distributed event streaming platform built by the Apache Software Foundation.
- It lets applications and systems publish, subscribe, store, and process streams of records in real time.
- Kafka is designed for horizontal scalability, high availability, and powerful data handling.
- Widely used for real-time data pipelines, streaming analytics, event sourcing, and microservices integration.

HOW KAFKA WORKS (HIGH-LEVEL VIEW)



WHY ORGANIZATIONS USE KAFKA

- **High Throughput**
- **Scalability**
- **Durability**
- **Fault Tolerance**
- **Real-Time Processing**
- **Loose Coupling**

KEY TAKEAWAY Apache Kafka provides a reliable, scalable, high-performance platform to stream events in real time and build modern data-driven applications.

Introduction to Apache Kafka

Apache Kafka is a distributed event streaming platform designed for high-throughput, fault-tolerant, real-time data pipelines and streaming applications.

What is Kafka?

A distributed publish–subscribe messaging system that allows applications to publish, store, process, and subscribe to streams of records.

High Throughput
Handles millions of messages per second.

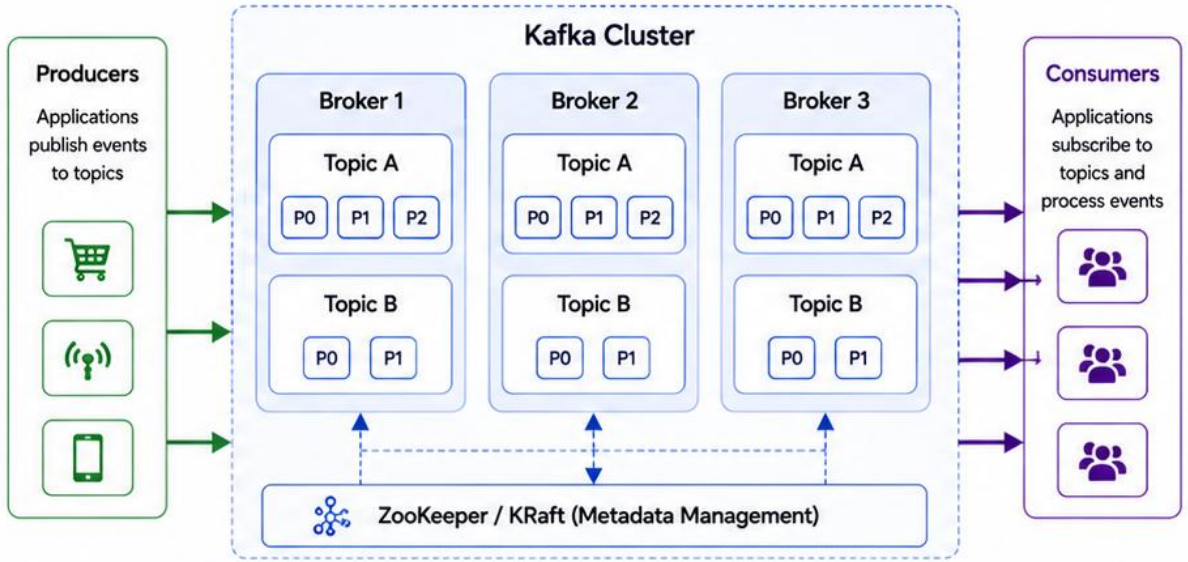
Scalability
Horizontally scalable and distributed by design.

Durability
Messages are persisted on disk and replicated.

Fault Tolerance
Replicated and resilient to node failures.

Real-Time Streaming
Built for real-time data streams and event processing.

Kafka Architecture



Common Use Cases

- Real-time Analytics
- Log Aggregation
- Data Integration
- Monitoring & Alerting
- Stream Processing
- Event Sourcing

Key Concepts

Topic
A category or feed name to which records are published.

Partition
A topic is divided into one or more partitions for scalability.

Broker
A Kafka server that stores data and serves clients.

Producer
An application that publishes events to Kafka topics.

Consumer
An application that subscribes to topics and processes events.

Consumer Group
A group of consumers that work together to consume from topics.

KAFKA ARCHITECTURE OVERVIEW

Kafka uses a publish-subscribe messaging model: data is published to topics, stored in partitions across brokers, and consumed by applications.

1

1. Producer

Sends records to a topic.

2

2. Broker (Leader)

Receives, writes to partition.

3

3. Replication

Record replicated to followers.

4

4. Consumer

Reads records at its own pace.

KEY ARCHITECTURAL CONCEPTS

Concept	What It Means
Topics	Categories or feeds to which records are published.
Partitions	Each topic is split into partitions for scalability and parallelism.
Replication	Partitions are replicated across brokers to ensure fault tolerance.
Leader & Followers	Each partition has one leader (reads/writes) and replicas (followers).
Retention	Events are retained for a configurable period, for replay and recovery.

WHY THIS ARCHITECTURE?

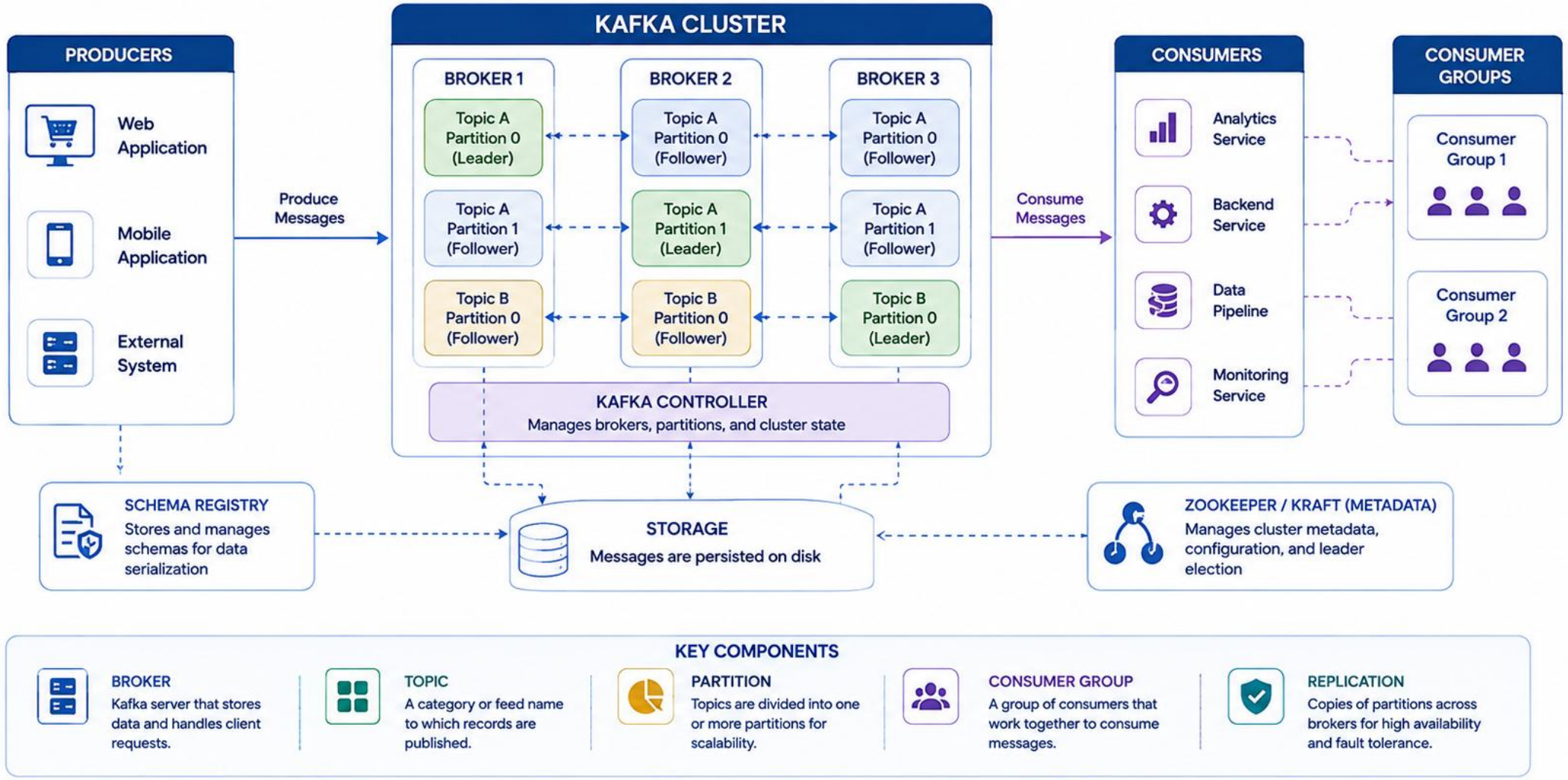
- Distributes load across many brokers and partitions.
- Provides high availability and fault tolerance.
- Enables horizontal scalability and high throughput.
- Supports real-time processing and replay of events.

CORE PRINCIPLE

A Kafka cluster is a group of one or more brokers that work together to store and serve data.

KEY TAKEAWAY Kafka's architecture -- topics, partitions, replication, and consumer groups -- enables it to deliver durable, scalable, and real-time data streaming for modern event-driven systems.

Kafka Architecture Overview



KAFKA BROKERS

Kafka brokers are the backbone of a Kafka cluster. They store data, serve client requests, and handle replication for high availability and durability.

KEY RESPONSIBILITIES

- **Store Data —** brokers store the records of topics in partitions on local disk.
- **Serve Clients —** handle read and write requests from producers and consumers.
- **Replication —** replicate partition data to other brokers for fault tolerance.
- **Partition Leadership —** each partition has one leader broker that handles all reads and writes.
- **Manage Metadata —** maintain metadata about topics, partitions, offsets, and the cluster.

TYPES OF BROKERS IN A CLUSTER

Type	Role
Leader Broker	Handles all read/write requests for a partition. One leader per partition.
Follower Broker	Replicates data from the leader; can become leader if it fails.
Controller Broker	Manages partition leadership elections and cluster metadata (KRaft mode in newer Kafka).

KEY CHARACTERISTICS

Scalability **Fault Tolerance** **High Availability**

Distributed storage: add more brokers to a cluster to handle more data and higher throughput. If one broker fails, replicas on other brokers keep data available.

KEY TAKEAWAY Kafka brokers store data, serve clients, and replicate partitions to ensure high availability and durability. Together, they form a resilient and scalable event streaming platform.

Kafka Brokers

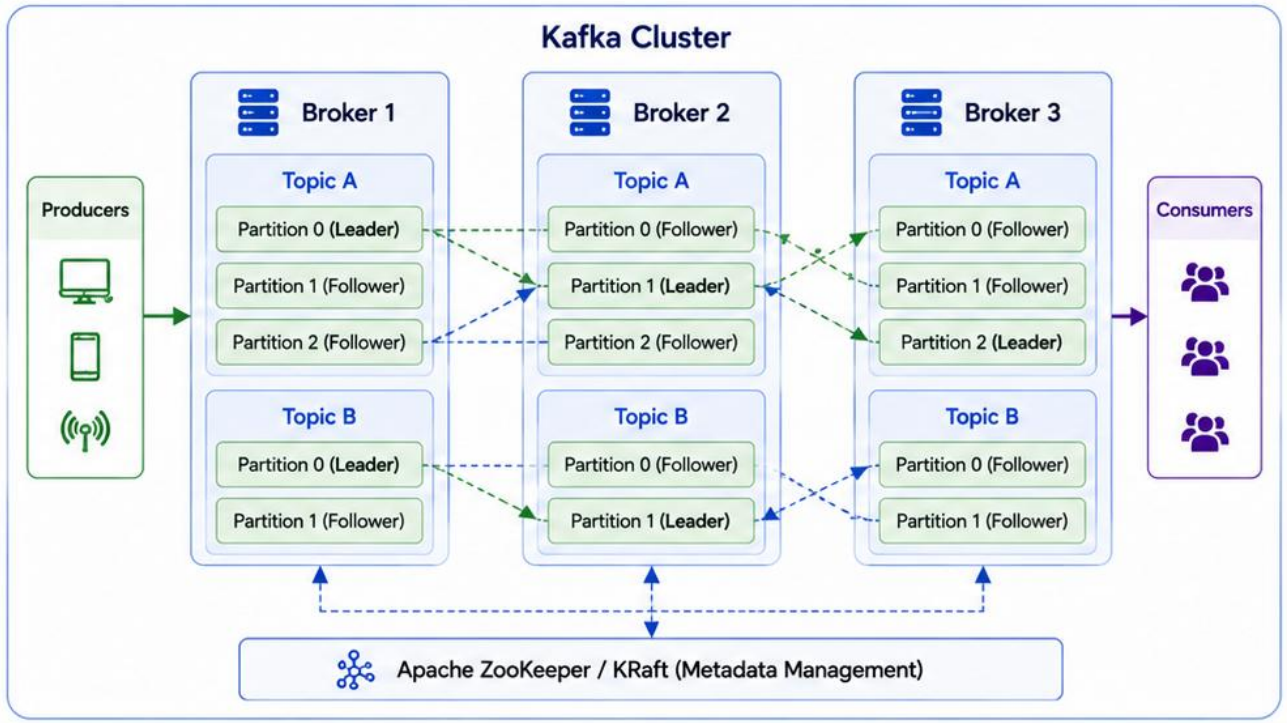
Kafka Brokers are the servers that store data, handle client requests, and replicate data across the cluster for durability and high availability.

What is a Kafka Broker?

A Kafka broker is a server in a Kafka cluster that stores topic partitions, serves clients, and manages data replication.

Key Responsibilities

- Store and manage topic partitions
- Replicate partitions across brokers
- Handle produce and consume requests
- Ensure data durability and availability
- Monitor and manage cluster health



Benefits of Kafka Brokers

- **High Availability** — Replicated data ensures no single point of failure
- **Scalability** — Add more brokers to handle more data
- **High Throughput** — Distribute load across multiple brokers
- **Durability** — Data is replicated and persisted on disk
- **Fault Tolerance** — Automatic leader election and recovery

Important Points

- Brokers work together as a cluster to provide fault tolerance.
- Each partition has one leader and multiple followers.
- If a leader broker fails, a follower is promoted as the new leader.
- Brokers store data on disk and keep it until the retention period expires.
- More brokers = higher throughput, scalability, and availability.

KAFKA TOPICS

A Kafka topic is a logical channel or feed name to which records are published. Topics are divided into partitions for scalability and parallelism.

KEY CHARACTERISTICS

- **Logical Feed —** topics represent a stream of related events or data.
- **Partitioned —** each topic splits into one or more partitions for parallelism and scalability.
- **Durable —** records are persisted on disk and retained for a configurable period.
- **Ordered Within Partition —** records within a partition are stored in order; ordering is not guaranteed across partitions.
- **Many-to-Many —** multiple producers can publish to a topic and multiple consumers can subscribe to it.

COMMON TOPIC CONFIGURATIONS

- **Number of Partitions —** determines parallelism; more partitions = more throughput and scalability.
- **Replication Factor —** number of copies of each partition across brokers (e.g. RF=3).
- **Retention Period —** how long records are retained, by time (e.g. 7 days) or size (e.g. 10 GB).
- **Cleanup Policy —** what happens to old data: delete (default) or compact (key-based data).

EXAMPLES OF TOPICS

orders **payments** **inventory-updates** **user-activity**

notifications

KEY TAKEAWAY Kafka topics organize data streams into logical feeds. Partitioning and replicating them across the cluster enables scalable, durable, reliable event streaming.

Kafka Topics

A Kafka Topic is a named feed of records to which data is published.
Topics are divided into partitions for scalability and parallelism.

What is a Topic?

A topic is a category or feed name to which producers publish records and consumers subscribe to read records.

Key Characteristics

Named Feed
Topics are uniquely identified by a name.

Partitioned
Topics are split into one or more partitions.

Scalable
More partitions = more parallelism and throughput.

Durable
Records are persisted and replicated for reliability.

Subscribed
Consumers subscribe to topics to read data.

Kafka Cluster

Benefits of Using Topics

High Throughput
Parallel processing across partitions.

Scalability
Add more partitions as data grows.

Flexibility
Multiple consumers can subscribe independently.

Isolation
Different topics isolate different data domains.

Reliability
Replicated partitions ensure data safety.

Important Points

- Topics can have multiple partitions distributed across brokers.
- Each partition is an ordered, immutable sequence of records.
- Records within a partition are consumed in the order they are written.
- Partition key determines the partition to which a record is written.
- Consumers in the same consumer group share partitions of a topic.

TRY IT YOURSELF — EXERCISE 2: TOPIC AND KEY MAP

Reinforces: freezing Northstar topic names, partitions, and keying before any broker runs — an architecture exercise.

REFERENCE — NORTHSTAR TOPIC MAP

Concept	Northstar Choice
Main topic	crm.customer-events.v1
DLQ topic	crm.customer-events.v1.dlq
Partitions (lab)	3
Record key	customerId (e.g. CUS-1001)

STEPS (IN notes/lab30-topic-map.md)

Step	What To Do
1 — Copy the Table	Recreate the reference table above in your notes; leave one blank row for a future Account* event topic name you invent.
2 — Keying Reason	Write why keying by CUS-1001 / CUS-1002 keeps a customer's events ordered within a single partition.
3 — Versioning	Explain what the .v1 suffix buys the team when the CustomerCreated payload schema needs to change later.
4 — DLQ Trigger	List two failure cases (conceptual only) that should land a record on the DLQ topic instead of the main one.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-topic-map.md (workspace: examples/module-30-exercises).

KAFKA PARTITIONS

Partitions are the fundamental unit of parallelism in Kafka. Each partition is an ordered, immutable sequence of records assigned a sequential offset.

WHY PARTITIONS MATTER

- **Scalability —** more partitions means more parallelism, which means higher throughput.
- **Parallelism —** multiple consumers can read different partitions simultaneously.
- **Fault Tolerance —** partitions are replicated across brokers for high availability.
- **Ordering Guarantee —** order is guaranteed within a partition, not across partitions.
- **Data Distribution —** partitions distribute data and load across the cluster.

KEY CONCEPTS

Concept	What It Means
Offset	Each record in a partition has a unique, sequential offset.
Leader	One broker acts as leader for a partition; handles all reads/writes.
Followers	Other brokers replicate the partition data from the leader.
Replication Factor	Number of copies of each partition stored across brokers.
Immutability	Once written, records cannot be changed or deleted (only retained or expired).

PARTITION BEST PRACTICES

Choose the right partition count | **Keep partition count consistent** | **Key messages properly** | **Monitor partition lag**

KEY TAKEAWAY Partitions are the key to Kafka's scalability and performance -- they allow data to be distributed, replicated, and consumed in parallel, while maintaining order within each partition.

Kafka Partitions

Partitions split a topic into multiple, ordered, immutable sequences of records.
They enable scalability, parallelism, and high throughput.

What is a Partition?

A partition is an ordered, immutable sequence of records within a topic. Each partition has a unique ID.

Key Characteristics

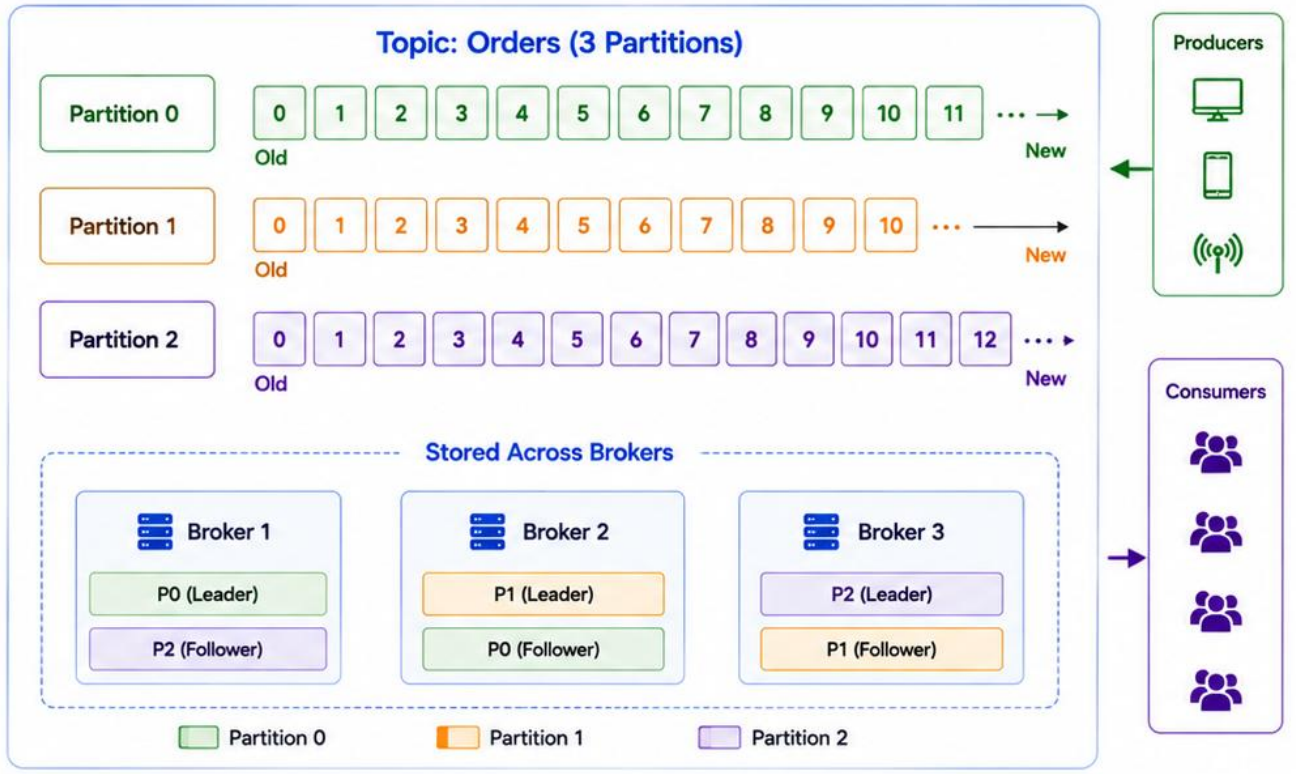
Ordered
Records within a partition are strictly ordered.

Immutable
Once written, records cannot be changed.

Scalable
More partitions allow more producers and consumers to operate in parallel.

Fault Tolerant
Partitions are replicated across brokers.

Independent Offsets
Each partition maintains its own offset per consumer.



Producers

Consumers

Benefits of Partitions

High Throughput
Parallel read/write across partitions.

Scalability
Scale producers and consumers horizontally.

Better Resource Utilization
Distribute load across multiple brokers.

Fault Tolerance
Replicated partitions ensure data safety.

Important Points

- A topic can have one or more partitions.
- Each partition has one leader and zero or more followers.
- Leaders handle all reads and writes for the partition.
- Followers replicate data from the leader.
- If a leader fails, a follower is promoted to leader.
- Partition count cannot be decreased (only increased).
- More partitions = more parallelism and throughput.

KAFKA REPLICATION

Replication in Kafka ensures that data is copied across multiple brokers (replicas) for high availability, fault tolerance, and durability.

WHY REPLICATION MATTERS

- **High Availability —** if a broker fails, replicas on other brokers keep serving the data.
- **Fault Tolerance —** replicas ensure data is not lost even if one or more brokers go down.
- **Data Durability —** data is safe and persistent across the cluster.
- **Read Scalability —** consumers can read from leader or follower replicas (with configuration).
- **Zero Data Loss (RF ≥ 3) —** replication factor 3 gives strong protection and durability.

KEY POINTS

- **Replication Factor (RF) —** number of copies of each partition; recommended RF=3 for production.
- **In-Sync Replicas (ISR) —** the subset of replicas that are fully caught up with the leader.
- **Leader Election —** if a leader fails, an in-sync replica is elected as the new leader.
- **Durability Guarantee —** data is committed once replicated to the required number of replicas (controlled by acks).

1. **Producer sends record**
2. **Leader writes to log**
3. **Followers replicate**
4. **Acknowledgment (per acks)**

KEY TAKEAWAY Kafka replication is the foundation of reliability. By storing multiple copies of partitions across brokers, Kafka ensures high availability, fault tolerance, and zero or minimal data loss.

Kafka Replication

Kafka replicates partitions across multiple brokers to ensure data durability, high availability, and fault tolerance.

What is Replication?

Replication means making copies of partition data on multiple brokers.

It protects data from broker failures.

Key Points

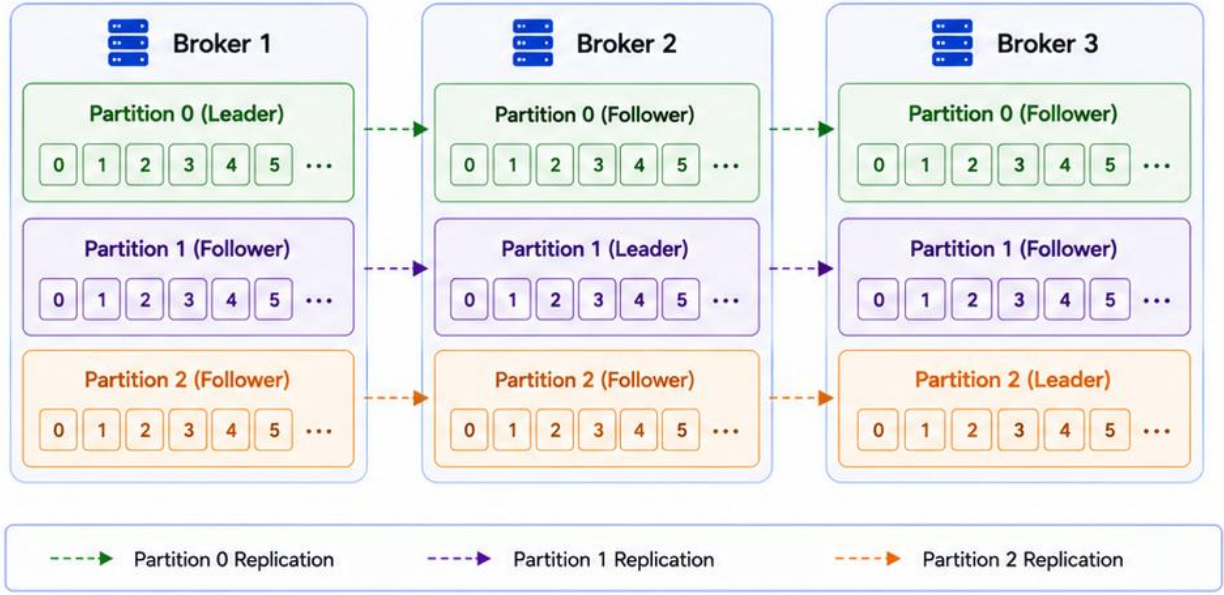
- Each partition has one leader and one or more followers.
- All writes go to the leader.
- Followers replicate data from the leader.
- If the leader fails, a follower is elected as the new leader.
- Replication factor (RF) defines the number of copies of each partition.

Example

Replication Factor (RF) = 3

Each partition is replicated on 3 brokers.

Topic: Orders (Replication Factor = 3)



Leader Failure Scenario

1. **Leader Fails**
   Leader of a partition (e.g., Partition 1 on Broker 2) fails.
2. **Election**
   Kafka elects one of the in-sync followers as the new leader.
3. **New Leader**
   The new leader starts serving reads and accepting writes.
4. **Replication Continues**
   Other followers continue replicating data from the new leader.

Replication Factor (RF)

Defines how many copies of each partition are maintained.

Common values: 1, 2, 3, ...

Recommended: RF = 3

- Survives one broker failure.
- High availability.
- No data loss.

Benefits

- **Fault Tolerance**
  Data is safe even if a broker fails.
- **High Availability**
  Partitions remain available during failures.
- **Data Durability**
  Reduces risk of data loss.
- **Scalability**
  Distribute load and data across brokers.

KAFKA PRODUCERS AND CONSUMERS

Kafka follows a publish-subscribe model. Producers publish records to Kafka topics; consumers subscribe to topics and consume records at their own pace.

PRODUCERS

- **Publish Records —** send data (records) to a specific topic.
- **Choose Partition —** decide which partition a record goes to (key-based or round-robin).
- **Reliability (Acks) —** control durability with acknowledgments: acks=0, 1, or all.
- **Batching & Compression —** improve performance by batching records and compressing them.

CONSUMERS

- **Subscribe to Topics —** consumers subscribe to one or more topics.
- **Read in Order (Per Partition) —** records within a partition are read in exact order.
- **Consumer Groups —** multiple consumers can work together to parallelize consumption.
- **Offset Management —** track progress using offsets; supports auto or manual commit.

Producer config (acks=all, idempotent)

```
props.put(ACKS_CONFIG, "all");
props.put(ENABLE_IDEMPOTENCE_CONFIG, true);

var record = new ProducerRecord<>(
    TOPIC, customerId, json);
var m = producer.send(record).get();
```

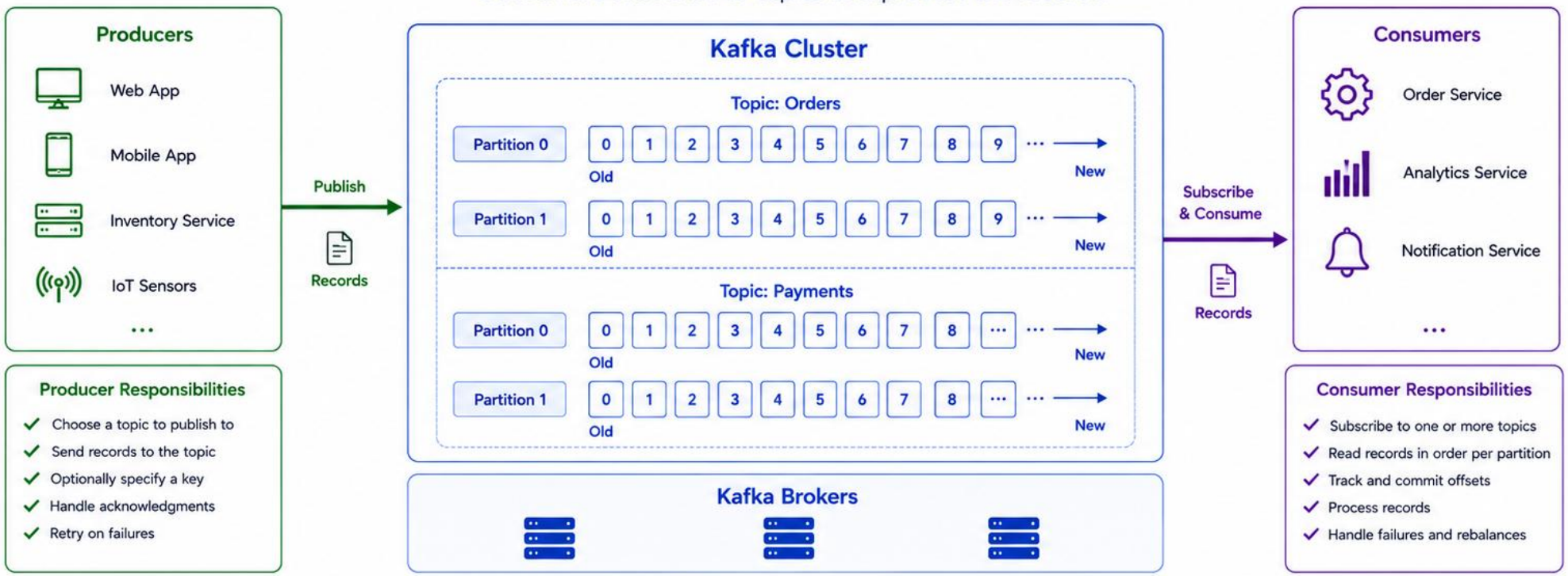
BEST PRACTICES

- Choose good keys
- Batch & compress
- Monitor lag
- Idempotent producers
- Stateless, resilient consumers

KEY TAKEAWAY Producers publish data to Kafka topics. Consumers subscribe and process that data at their own pace. Kafka's design enables scalable, reliable, real-time data streaming.

Kafka Producers and Consumers

Producers publish records to Kafka topics.
Consumers subscribe to topics and process the records.



Producer Responsibilities

- ✓ Choose a topic to publish to
- ✓ Send records to the topic
- ✓ Optionally specify a key
- ✓ Handle acknowledgments
- ✓ Retry on failures

Consumer Responsibilities

- ✓ Subscribe to one or more topics
- ✓ Read records in order per partition
- ✓ Track and commit offsets
- ✓ Process records
- ✓ Handle failures and rebalances

Producer Flow

Create Record → (Optional) Add Key → Send to Kafka Topic → Acknowledgment (ACK)

Key Concepts

- Producers write data to topics.
- Consumers read data from topics.
- Data is immutable and ordered within partitions.
- Multiple producers and consumers enable scalability.

Consumer Flow

Subscribe to Topic(s) → Kafka Assigns Partitions → Poll & Read Records → Process Records → Commit Offsets

TRY IT YOURSELF — EXERCISE 3: EVENT ENVELOPE SKETCH

Reinforces: drafting versioned CustomerCreated / CustomerStatusChanged JSON envelopes for Amina and Ravi — a documentation exercise.

ENVELOPE FIELDS

Shared envelope shape (headers + payload)

```
{ "eventType": "CustomerCreated", "eventVersion": 1,
  "occurredAt": "<ISO-8601 timestamp>", "correlationId": "lab-request-001",
  "customerId": "CUS-1001", "payload": { /* event-specific fields */ } }
```

STEPS (IN notes/lab30-envelopes.md)

Step	What To Do
1 — Headers	List the shared envelope fields you will reuse for every event: eventType, eventVersion, occurredAt, correlationId, customerId, payload.
2 — Amina Sample	On paper, sketch a CustomerCreated envelope for CUS-1001 Amina Khan with correlationId=lab-request-001.
3 — Ravi Sample	Sketch a CustomerStatusChanged envelope for CUS-1002 Ravi Singh (e.g. PROSPECT to ACTIVE, or ACTIVE to SUSPENDED).
4 — Compatibility Note	Write one rule: consumers must ignore unknown payload fields — the envelope is forward-compatible, not fixed.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-envelope-sketch.md (workspace: examples/module-30-exercises).

KAFKA CONSUMER GROUPS

Consumer groups let multiple consumers work together to parallelize consumption of a topic's partitions. Each partition is consumed by one consumer within a group.

WHY CONSUMER GROUPS?

- **Parallelism —** distribute partitions across multiple consumers to increase throughput.
- **Scalability —** add more consumers to a group to handle more partitions and more data.
- **Fault Tolerance —** if a consumer fails, its partitions are reassigned to other members.
- **Offset Management —** offsets are tracked per group, so each group has independent progress.
- **Multiple Processing Views —** different consumer groups can read the same topic independently.

EXAMPLE: TOPIC WITH 3 PARTITIONS

Group Size	Assignment
1 consumer in group	One consumer reads all 3 partitions.
2 consumers in group	Partitions are distributed across consumers.
3 consumers in group	Each consumer reads exactly one partition.

KEY CONCEPTS

- **Rebalancing —** when consumers join, leave, or fail, Kafka rebalances partitions among survivors.
- **At-Least-Once by Default —** consumers commit offsets after processing so no data is lost.

KEY TAKEAWAY Consumer groups enable scalable, fault-tolerant, efficient data consumption. Multiple consumers can work together without duplicate processing, while each group maintains independent progress.

Kafka Consumer Groups

A consumer group is a set of consumers that work together to consume data from topics. Each partition is consumed by only one consumer in the group.

What is a Consumer Group?

A consumer group is a group of related consumers that coordinate to read from topics.

Key Characteristics

Parallel Consumption
Partitions are distributed among consumers in the group.

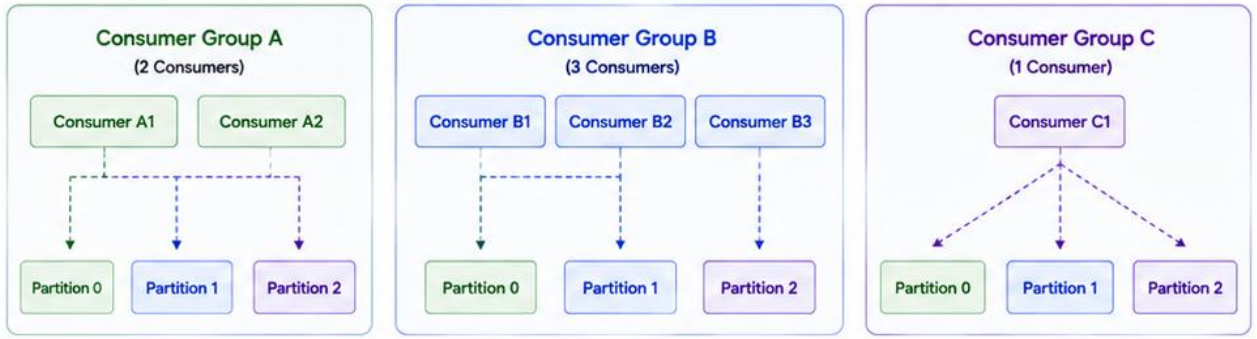
Load Balancing
Consumers share the work of consuming partitions.

Fault Tolerance
If a consumer fails, its partitions are reassigned to other consumers.

Independent Progress
Each group maintains its own offset.

Scalability
Add more consumers to increase throughput.

Topic: Orders (3 Partitions)



How It Works

Join Group
Consumers join the same consumer group.

Rebalance
Kafka assigns partitions to consumers in the group.

Consume
Consumers read records from their assigned partitions.

Rebalance on Change
If a consumer joins or leaves, partitions are reassigned.

Fault Tolerance
If a consumer fails, partitions are reassigned to others.

Example

- 3 Partitions, 2 Consumers in Group
  → One consumer is idle.
- 3 Partitions, 3 Consumers in Group
  → Each consumer gets one partition.
- 3 Partitions, 5 Consumers in Group
  → Two consumers are idle.

Important Points

- Within a group, each partition is consumed by only one consumer.
- Different consumer groups can read the same topic independently.
- Offsets are maintained per consumer group.
- If consumers leave or fail, partitions are automatically rebalanced.
- More consumers in a group can increase parallelism (up to the number of partitions).

TRY IT YOURSELF — EXERCISE 4: FILL KAFKA BASICS TODOS (STARTER)

Reinforces: recalling topic, partition, offset, and consumer-group vocabulary — a fill-in-the-blank starter, not a finished solution.

STARTER QUIZ (COPY INTO notes/lab30-kafka-todos.md)

TODO — replace every blank

```
1. A _____ is a named stream of records.  // TODO
2. A _____ is an ordered subset of a topic; offsets are per partition.  // TODO
3. The _____ is the consumer's position in a partition.  // TODO
4. Consumers in the same _____ compete for partitions;
   different groups each get a copy.  // TODO
```

STEPS

Step	What To Do
1 — Copy the Quiz	Create notes/lab30-kafka-todos.md and paste the four-blank starter quiz shown above.
2 — Fill Blanks	Replace each ____ with exactly one of: topic / partition / offset / consumer group. Reason through the definition before you write it.
3 — CRM Example	Add one line: group crm-notifications shares partitions; group crm-audit reads all CUS-1001/CUS-1002 events independently.
4 — Self-Check	Compare your four answers to the vocabulary covered on the last two slides: topic, partition, offset, consumer group.

KEY TAKEAWAY TRY IT NOW (STARTER) — Complete Exercise 4 before continuing: exercises/exercise-04-fill-kafka-basics.md — replace every ____ and // TODO before moving on (workspace: examples/module-30-exercises).

KAFKA MESSAGE ORDERING

Kafka guarantees ordering of messages within a partition, not across partitions. If order matters, ensure related messages are written to the same partition.

KEY TAKEAWAYS

- **Order Is Guaranteed —** Kafka guarantees strict message order within a single partition.
- **Not Across Partitions —** there is no ordering guarantee across partitions or across topics.
- **Key for Ordering —** use the same key when producing so related messages go to the same partition.
- **Single Consumer —** only one consumer in a group reads a given partition, preserving its order.
- **Offsets Ensure Order —** consumers read messages in increasing offset order within a partition.

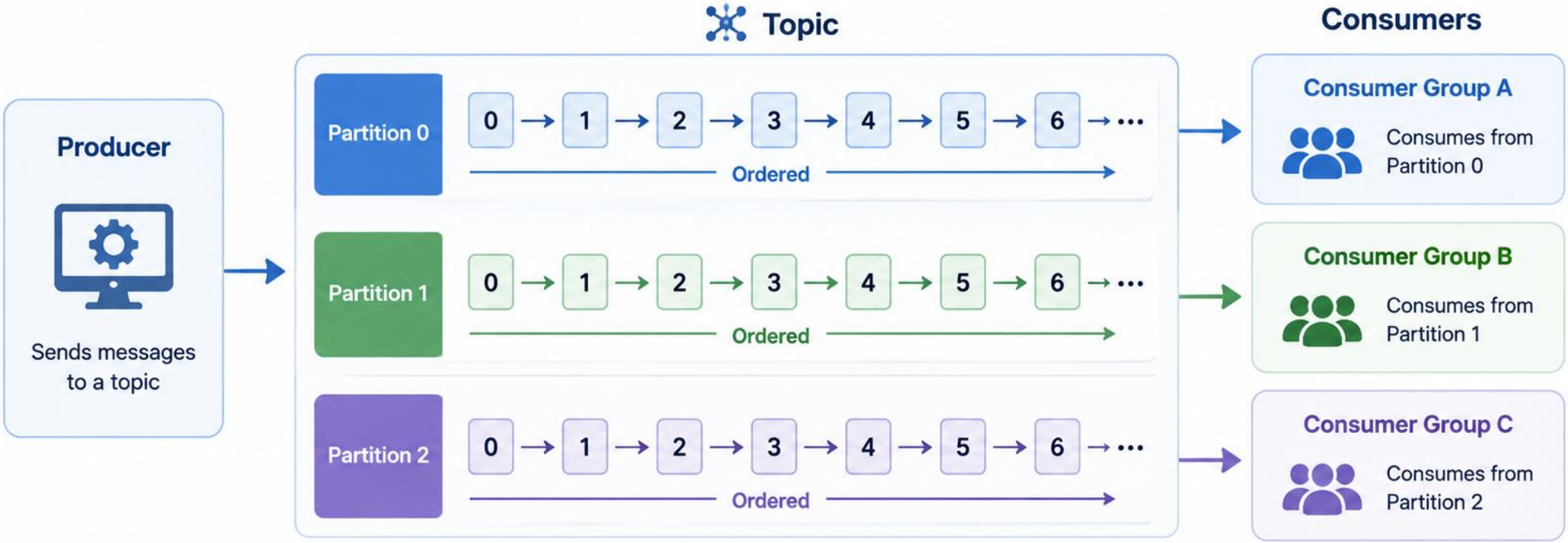
BEST PRACTICES FOR ORDERING

- **Use Keys Wisely —** use a meaningful key (e.g. userId, customerId) so related messages land together.
- **Control Partition Count —** more partitions = more parallelism, but harder to maintain global order.
- **Single Consumer per Partition —** Kafka ensures only one consumer in a group reads a partition.
- **Avoid Repartitioning —** changing partition count can break existing ordering guarantees.
- **Design for Idempotency —** handle duplicates gracefully, since at-least-once delivery may occur.

KEY TAKEAWAY Kafka guarantees message ordering within a partition by offset. To maintain order for related events, send them with the same key to the same partition, using a single consumer within a group.

Message Ordering

Messages within a partition are ordered.
Messages across partitions are not guaranteed to be ordered.



Within a Partition
Messages are guaranteed to be ordered
(0 → 1 → 2 → 3 → ...)

Across Partitions
No ordering guarantee
(messages may arrive in any order)

TRY IT YOURSELF — EXERCISE 5: PRODUCER CHECKLIST

Reinforces: planning reliable Lab 30 producer settings — acks, idempotence, and keying — before writing any code.

CHECKLIST (LAB 30 JAVA PRODUCER)

Setting	Why It's There
acks = all	Wait for every in-sync replica to acknowledge before considering the CRM event durable.
Idempotent producer	The broker dedupes producer retries, so Amina is not accidentally double-created in the log.
key = customerId	Guarantees per-customer ordering by routing every event for one customer to the same partition.
value = JSON envelope	The versioned CustomerCreated / CustomerStatusChanged envelope sketched in Exercise 3.

STEPS (IN notes/lab30-producer-checklist.md)

Step	What To Do
1 — Settings List	Write the checklist: acks=all, idempotent producer, key=customerId, value=JSON envelope.
2 — Why acks=all	One sentence: wait for the ISR (in-sync replica) acknowledgment before considering the CRM event durable.
3 — Idempotence	One sentence: the broker dedupes producer retries so Amina is not double-created in the log.
4 — Out of Scope Today	Write and mark explicitly: 'Do not run kafka-console-producer in this pre-lab.'

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-producer-checklist.md (workspace: examples/module-30-exercises).

KAFKA MESSAGE RETENTION

Message retention defines how long Kafka retains records in a topic before deleting them -- managing storage, controlling data lifecycle, and meeting compliance needs.

WHY MESSAGE RETENTION MATTERS

- **Manage Storage —** prevent unlimited growth of data and manage disk space efficiently.
- **Cost Optimization —** reduce storage and infrastructure costs by retaining only what's necessary.
- **Compliance —** retain data for a required period to meet legal and regulatory obligations.
- **Data Lifecycle —** automatically remove stale data based on business needs.
- **Improved Performance —** smaller data sets lead to faster replication and recovery.

RETENTION CONFIGURATION OPTIONS

- **Time-Based Retention —** based on how long messages are kept (retention.ms, e.g. 7 days).
- **Size-Based Retention —** based on the total size of data in a topic/partition (retention.bytes).
- **Cleanup Policy —** delete (default, remove old records) or compact (keep only the latest per key).

Scenario	Result
Both time and size are set	Deleted by whichever limit is hit first.
Only time is set	Messages deleted based on age.
Only size is set	Messages deleted based on total size.
Neither is set	Kafka default retention applies (7 days).

KEY TAKEAWAY Message retention in Kafka helps control storage, costs, and data lifecycle by automatically deleting old messages based on time or size. Retention is applied independently on each partition.

Message Retention

Kafka retains messages for a configurable period or size before they are automatically deleted.

What is Message Retention?

Message retention defines how long Kafka keeps messages in a topic before deleting them.

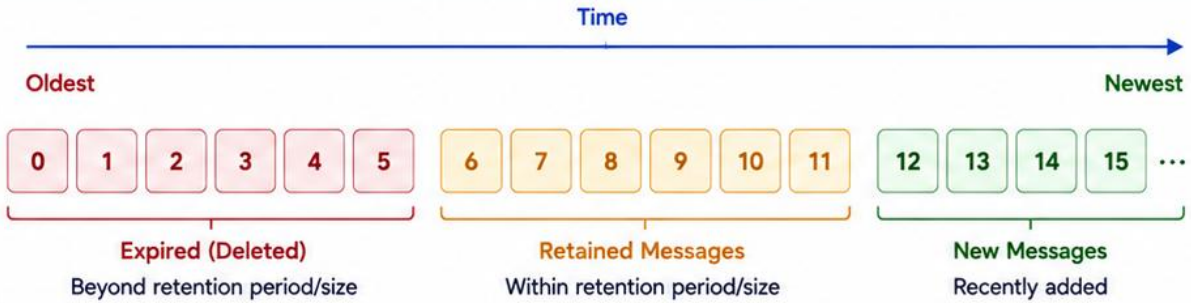
Retention Configuration

Time-based Retention
retention.ms
e.g., 7 days

Size-based Retention
retention.bytes
e.g., 10 GB

Both can be set together. The first limit reached will trigger deletion.

Topic: Orders (Partition 0)



Kafka automatically deletes expired messages.

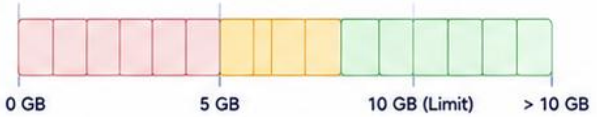
Cleanup is performed by the log cleaner based on retention policy.

Retention Scenarios

1. Time-based Retention (retention.ms = 7 days)

Messages older than 7 days are deleted.

2. Size-based Retention (retention.bytes = 10 GB)



When size exceeds 10 GB, oldest messages are deleted.

Key Points

- Ensures disk space is managed effectively.
- Helps meet compliance and data governance requirements.
- Deleted messages cannot be recovered.
- Retention is independent of consumer groups.

Benefits

- Prevents unlimited storage growth.
- Reduces infrastructure costs.
- Improves system performance.

KAFKA EVENT PROCESSING FLOW

Kafka enables real-time event streaming where events are produced, stored, and consumed by applications that process and act on the data.

1. **Event Source**
   Apps, devices, DBs, logs.
2. **Producer**
   Serializes & publishes.
3. **Kafka Cluster**
   Stores as an event log.
4. **Consumers**
   Read at their own pace.
5. **Processing & Action**
   Enrich, update, alert.

KEY BENEFITS & CHARACTERISTICS

- **Real-Time —** insights and actions within seconds; events are processed as they happen.
- **High Throughput —** handles high volumes with ease and low latency.
- **Reliable & Decoupled —** fault-tolerant, highly available; producers and consumers are independent.
- **Scalable —** scales horizontally with demand as event volume grows.

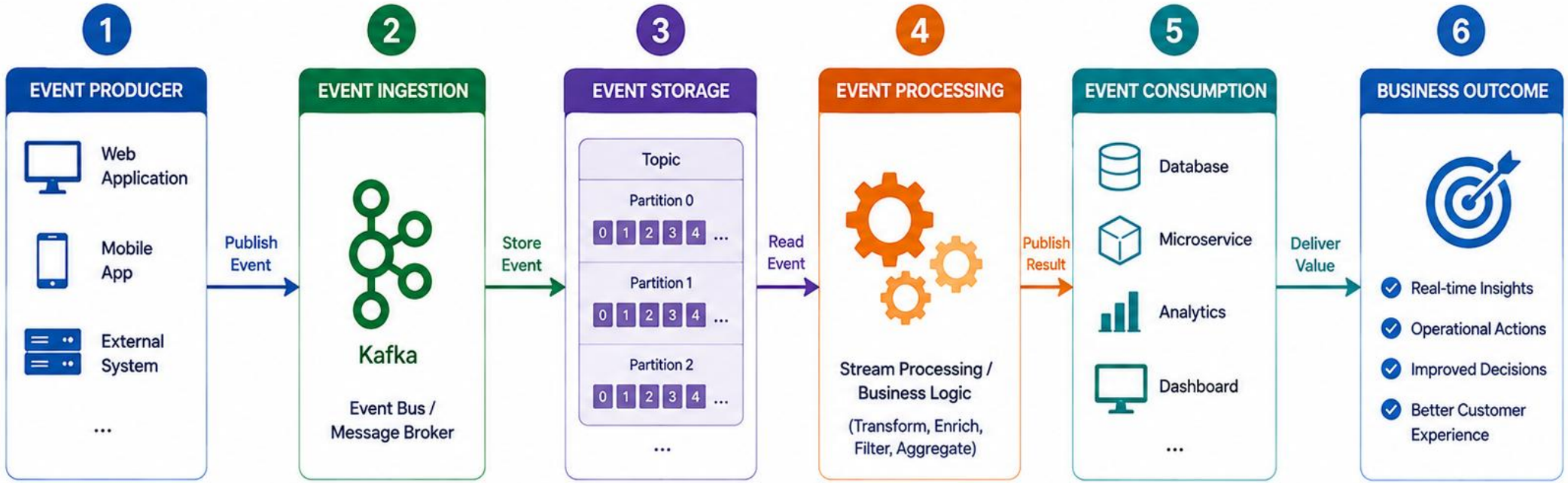
COMMON USE CASES

- Real-time analytics dashboards.
- Fraud detection and alerting.
- IoT data ingestion and monitoring.
- Microservices communication.
- Log aggregation and monitoring.

KEY TAKEAWAY Kafka provides a powerful event streaming platform that enables real-time, scalable, and reliable event processing. It decouples systems and lets organizations build responsive, data-driven applications.

Event Processing Flow

Events are produced, captured, processed, and consumed to deliver business value.



KEY CHARACTERISTICS

REAL-TIME
Events are processed as they occur

LOOSE COUPLING
Producers and consumers are independent

SCALABLE
Handle high volume with ease

RELIABLE
Durable storage and fault tolerance

FLEXIBLE
Easy to adapt to new requirements

DECOUPLING SERVICES WITH EVENTS

Kafka enables services to communicate asynchronously through events, reducing tight coupling and increasing flexibility, scalability, and resilience.

	Traditional (Coupled)	Event-Driven (Decoupled)
Communication	Direct, blocking calls between services.	Publish an event; subscribers react asynchronously.
Failure Impact	If any service is slow or down, the entire flow is impacted.	Services communicate through events; they stay independent and resilient.
Evolution	Adding a consumer requires changing the caller.	Add new consumers by subscribing -- no producer changes needed.

HOW DECOUPLING WORKS WITH EVENTS

- **1. Event Producer —** publishes an event to Kafka when something important happens.
- **2. Event Store (Kafka) —** stores events durably and makes them available.
- **3. Event Consumers —** subscribe to relevant events and process them asynchronously.
- **4-6. Independent Evolution —** producers/consumers evolve independently; delivery is reliable and replayable; multiple consumers can each serve a different purpose.

BENEFITS OF DECOUPLING WITH EVENTS

Loose coupling

Better fault isolation

Easier integration of new services

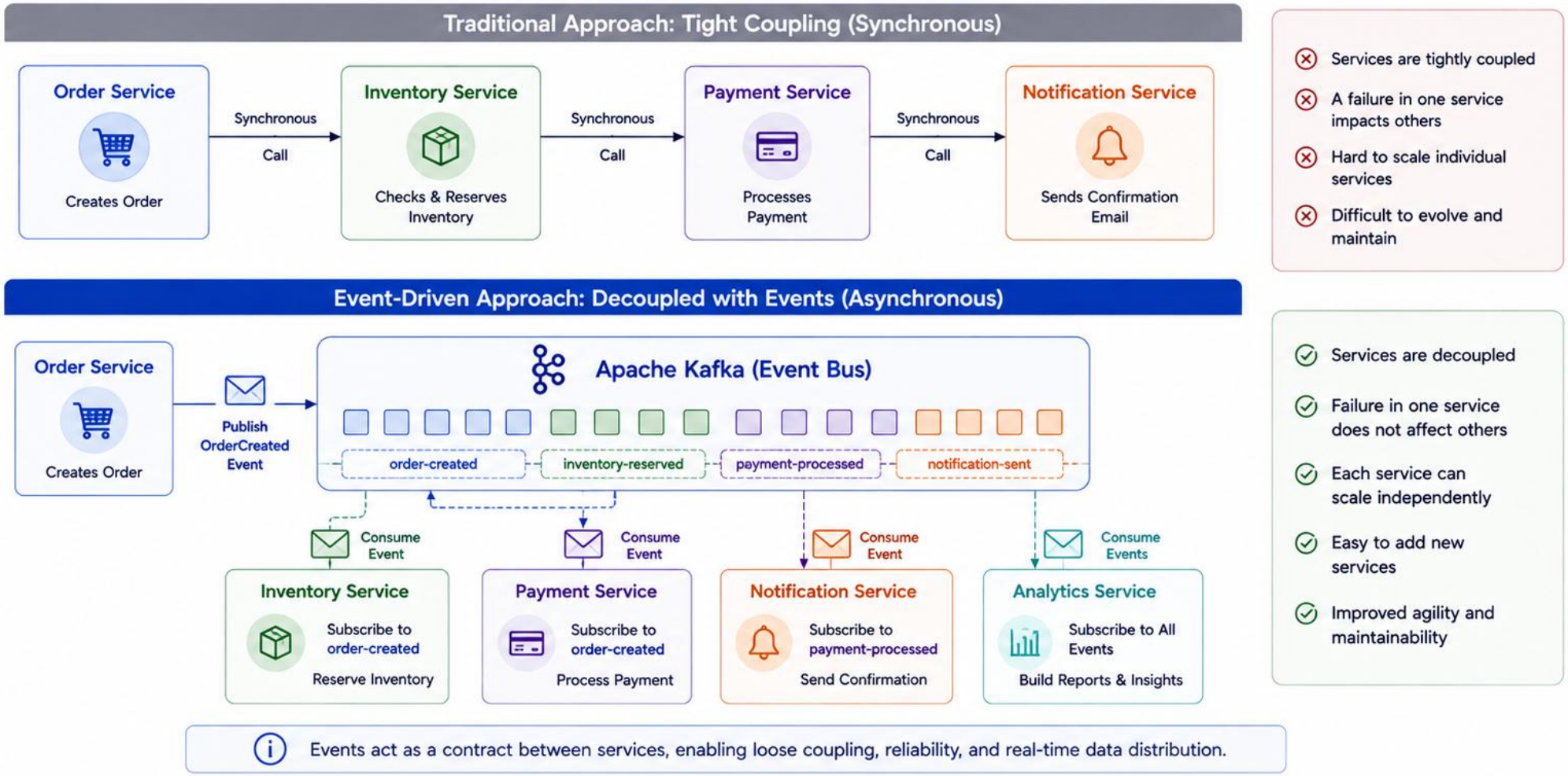
Event replay & audit

Improved agility & maintainability

KEY TAKEAWAY Decoupling services with events using Kafka reduces dependencies, improves resilience and scalability, and enables systems to evolve quickly and respond to business needs.

Decoupling Services with Events

Events help services communicate asynchronously, reducing dependencies and improving scalability.



- ⊗ Services are tightly coupled
- ⊗ A failure in one service impacts others
- ⊗ Hard to scale individual services
- ⊗ Difficult to evolve and maintain

- ✓ Services are decoupled
- ✓ Failure in one service does not affect others
- ✓ Each service can scale independently
- ✓ Easy to add new services
- ✓ Improved agility and maintainability

REAL-TIME DATA PROCESSING WITH KAFKA

Kafka enables real-time data streaming and processing so organizations can analyze events, detect patterns, and take action instantly.

1. **Data Sources**
   Apps, IoT, web, DBs, logs.
2. **Ingest & Stream**
   Published to Kafka topics.
3. **Stream Processing**
   Flink, Kafka Streams, ksqlDB.
4. **Outputs & Actions**
   Dashboards, alerts, DBs.
5. **Downstream**
   BI, ML, automated workflows.

WHY REAL-TIME PROCESSING MATTERS

- **Instant Insights —** get insights within seconds instead of minutes or hours.
- **Faster Decisions —** react to events immediately and automate responses.
- **Better Customer Experience —** deliver personalized, timely experiences.
- **Fraud & Anomaly Detection —** detect suspicious activities as they happen.
- **Operational Efficiency —** streamline operations and reduce manual effort.

REAL-TIME USE CASES

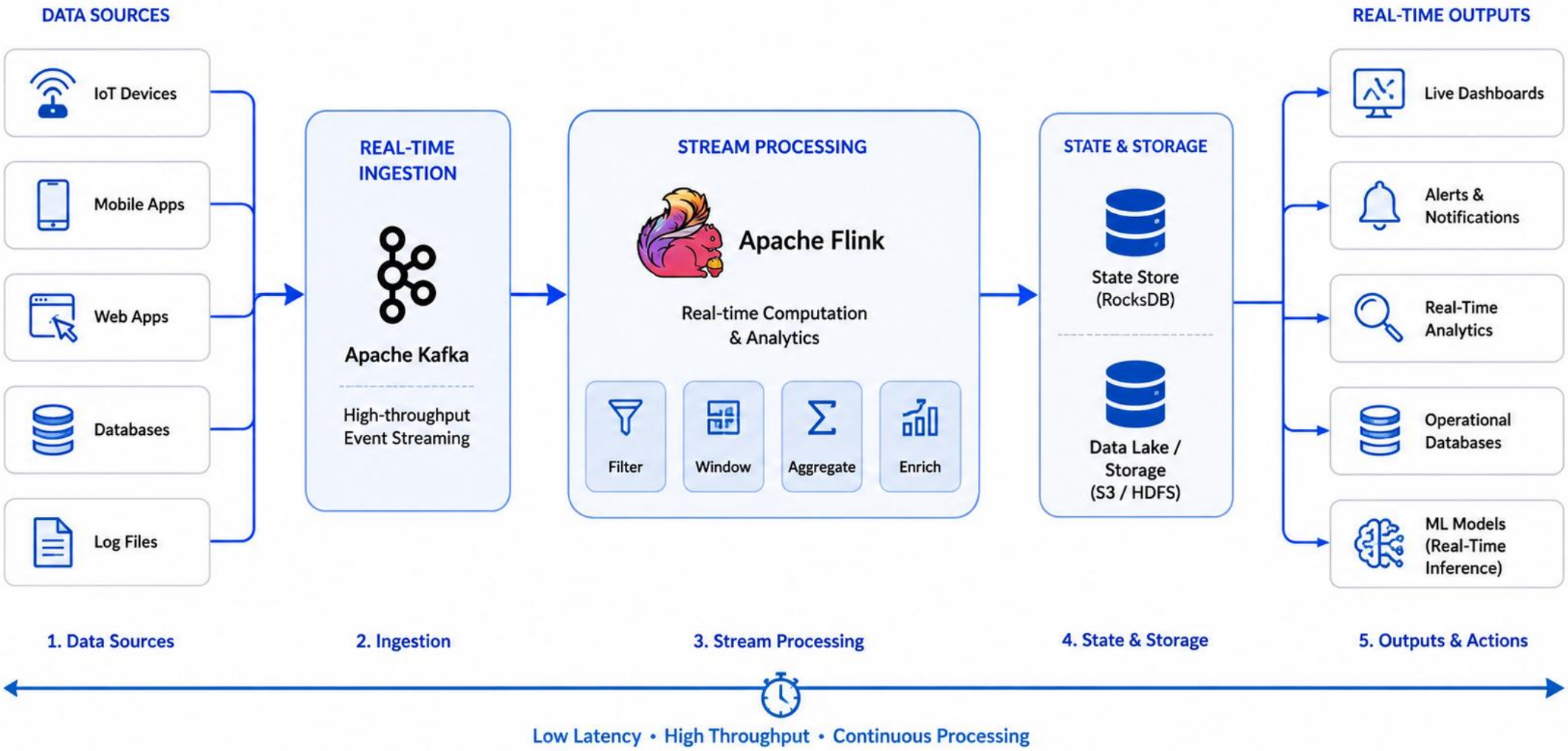
E-Commerce | **Fraud Detection** | **IoT Monitoring** | **Financial Services** | **Social Media Analytics**

EXAMPLE: REAL-TIME FRAUD DETECTION

Transaction occurs -> published to a Kafka topic -> stream processing (rules engine) scores it -> a fraud alert is triggered -- all within seconds.

KEY TAKEAWAY Kafka enables real-time data processing by capturing events as they happen, processing them continuously, and delivering results instantly.

Real-Time Data Processing



ENTERPRISE KAFKA USE CASES

Kafka is the backbone of event-driven architectures in enterprises, enabling real-time data movement, integration, and insights at scale across systems and teams.

Use Case	What It Enables
1. Real-Time Analytics	Stream events from apps, websites, and IoT devices for real-time dashboards (e.g. user behavior analytics on an e-commerce platform).
2. Fraud Detection	Analyze transactions as they occur to detect suspicious activity and trigger alerts (e.g. real-time credit card fraud detection).
3. Event-Driven Microservices	Decouple services by exchanging events (e.g. order created, payment processed, inventory updated).
4. Data Integration	Integrate disparate systems by streaming data between them (e.g. syncing CRM, ERP, and data warehouse systems).
5. IoT & Telemetry Data	Ingest high-volume telemetry from millions of devices for monitoring and predictive maintenance (e.g. smart factory sensors).
6. Log & Event Aggregation	Collect logs and application events for centralized monitoring and analysis (e.g. security monitoring and compliance).
7. Customer 360 View	Stream interactions from multiple touchpoints to build a unified customer profile (e.g. combining web, mobile, and support events).
8. Content Streaming	Deliver content updates, notifications, or live feeds in real time (e.g. live sports scores, stock market updates).
9. Workflow Orchestration	Coordinate complex business processes by streaming events between steps (e.g. insurance claim processing).

KEY TAKEAWAY Kafka empowers enterprises to build real-time, event-driven systems that are scalable, resilient, and future-ready -- driving innovation and better business outcomes.

Enterprise Kafka Use Cases

TRY IT YOURSELF — EXERCISE 6: LAB 30 READINESS

Capstone pre-lab self-check — confirm you know what evidence Lab 30 will ask for, without starting Kafka.

READINESS CHECK

Deliverable	What To Confirm
Topics	crm.customer-events.v1 (3 partitions) and its .dlq (1 partition) are named and frozen (Exercise 2).
Samples	Versioned CustomerCreated / CustomerStatusChanged JSON envelopes exist for Amina and Ravi (Exercise 3).
Producer	acks=all + idempotence + key=customerId checklist is ready to implement (Exercise 5).
Notes	A kafka-notes.md runbook plan is in place for the Lab 31 hand-off.

STEPS (SELF-CHECK, NO KAFKA YET)

Step	What To Do
1 — Deliverable Skim	From the Lab 30 guide header, list the four deliverable categories: topics, samples, producer, notes.
2 — Fixture Recall	Write CUS-1001 Amina, CUS-1002 Ravi, and correlation ID lab-request-001 from memory, without looking them up.
3 — Path Plan	Note the target folder idea java-bootcamp/examples/lab30-crm — do not create it yet if you would rather wait for the lab.
4 — Pass/Fail Preview	Write Pass if your notes cover topics + keys + envelopes; otherwise write Fail and revisit Exercises 2, 3, and 4 first.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before Lab 30: exercises/exercise-06-lab30-readiness.md (workspace: examples/module-30-exercises).

LAB OVERVIEW -- EVENT-DRIVEN ARCHITECTURE WITH KAFKA

Lab 30: Event-Driven Architecture with Kafka -- Northstar CRM Topics

BUSINESS SCENARIO

- After HTTP APIs (Labs 25-29), Northstar needs asynchronous fan-out for notifications and audit without coupling every consumer to the Customer service JVM.
- Leadership freezes topic names and keying strategy before Spring Kafka wiring in Lab 31: customer lifecycle facts are published as versioned events on crm.customer-events.v1, keyed by customerId, with a sibling DLQ topic for poison messages.
- You stand up a local Kafka KRaft broker and build the topic foundation -- no PostgreSQL or Resilience4j required yet.

LEARNING OBJECTIVES

- Create a local single-node Kafka broker with Docker Compose (KRaft).
- Create partitioned CRM event and dead-letter topics explicitly.
- Design versioned CustomerCreated and CustomerStatusChanged event envelopes.
- Publish customer-keyed records from Kafka CLI and Java.
- Compare competing versus independent consumer groups; inspect lag and replay.

WHAT YOU BUILD

- lab30-crm with a Compose KRaft broker; crm.customer-events.v1 (3 partitions) and crm.customer-events.v1.dlq (1 partition); versioned JSON envelopes for Amina/Ravi; a Java KafkaProducer with acks=all + idempotence; competing crm-notifications vs independent crm-audit consumer groups; a kafka-notes.md runbook for Lab 31 hand-off.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation ID lab-request-001 appears on every event envelope; topic names and keys are frozen for Lab 31.

LAB TASKS AND DELIVERABLES (1 OF 2)

Northstar CRM Topics -- the eight implementation steps, in build order.

#	Implementation Step
1	Scaffold lab30-crm; start Kafka in KRaft mode via Docker Compose.
2	Create crm.customer-events.v1 (3 partitions) and .dlq (1 partition) topics explicitly.
3	Write versioned CustomerCreated / CustomerStatusChanged JSON envelopes for Amina/Ravi.
4	Publish keyed events with the console producer (key=customerId).
5	Consume records showing key, partition, offset, and timestamp.
6	Build a Java KafkaProducer with acks=all + idempotence (ENABLE_IDEMPOTENCE_CONFIG).
7	Compare competing (crm-notifications) vs independent (crm-audit) consumer groups.
8	Inspect consumer lag and restart catch-up; document a kafka-notes.md runbook + production checklist.

Sample keyed produce line

```
CUS-1001:{"eventType":"CustomerCreated","eventVersion":1,
  "customerId":"CUS-1001","correlationId":"lab-request-001",
  "data":{"fullName":"Amina Khan","status":"ACTIVE"}}
```

KEY TAKEAWAY Steps 4 and 5 are deliberate: publish keyed events first, then consume and observe key/partition/offset/timestamp together to SEE Kafka's per-key ordering, not just be told about it.

LAB TASKS AND DELIVERABLES (2 OF 2)

Northstar CRM Topics -- expected deliverables and the 100-mark evaluation rubric.

EXPECTED DELIVERABLES

- compose.yaml KRaft broker definition.
- Topics created (events + DLQ) with describe evidence.
- Versioned event JSON samples (Amina/Ravi).
- CLI produce/consume evidence with keys/partitions/offsets.
- Competing vs independent group evidence + lag describe.
- kafka-notes.md runbook + production checklist.

RUBRIC (100 MARKS)

- **Environment —** 10 marks.
- **Core Implementation —** 30 marks.
- **Integration/Config —** 15 marks.
- **Failure Handling —** 15 marks.
- **Verification —** 10 marks.
- **Security/Prod Awareness —** 10 marks.
- **Docs —** 10 marks.

KEY TAKEAWAY Random keys that break per-customer ordering are incomplete; inventing topic names Lab 31 cannot reuse is a continuity failure.

MODULE 30 SUMMARY

In this module, you explored Apache Kafka and how it enables event-driven architectures for building scalable, real-time, and resilient systems.

KEY CONCEPTS COVERED

Kafka Fundamentals

Topics, partitions, brokers, replicas, producers, and consumers.

Consumer Groups

Parallel consumption, load balancing, and fault tolerance.

Ordering & Retention

Ordering guarantees within partitions and message retention policies.

Decoupling with Events

How Kafka decouples services and enables asynchronous communication.

Real-Time Processing

Stream processing, real-time analytics, and immediate actions.

Enterprise Use Cases

Real-world scenarios across industries driving business value.

MODULE FLOW RECAP

1. **Event Producers**
2. **Kafka Cluster**
3. **Event Consumers**
4. **Processing & Action**
5. **Lab: Northstar CRM**

KEY TAKEAWAY Kafka enables event-driven architectures that are scalable and resilient. Events allow systems to communicate asynchronously and independently. Events are the new API.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Apache Kafka fundamentals: what Kafka is, where it stores data, how topics are structured, and what ensures availability.

1. Apache Kafka is primarily a:

A. Relational database
B. Message queue
C. Distributed event streaming platform
D. File storage system

2. Which Kafka component stores messages on disk?

A. Producer
B. Consumer
C. Broker
D. Zookeeper

3. Topics in Kafka are divided into:

A. Tables
B. Partitions
C. Buckets
D. Schemas

4. What ensures high availability and fault tolerance in Kafka?

A. Single broker setup
B. Message encryption
C. Replication across brokers
D. Caching messages in memory

KEY TAKEAWAY Kafka is a distributed event streaming platform; brokers store messages on disk; topics are divided into partitions; replication across brokers ensures high availability and fault tolerance.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on consumer groups, retention, stream-processing tooling, and how Kafka enables decoupling between services.

5. Consumer groups in Kafka are used to:

A. Publish messages to topics

B. Scale out processing and load balance

C. Encrypt messages

D. Create topics

6. Message retention in Kafka is controlled by:

A. Partition count

B. Retention policy (time/size)

C. Consumer offset

D. Replication factor

7. Which tool is commonly used with Kafka for stream processing?

A. Hibernate

B. Apache Storm

C. Kafka Streams

D. JPA

8. Kafka enables decoupling between services by:

A. Sharing the same database

B. Using events as the contract

C. Tight synchronous communication

D. Direct API calls

KEY TAKEAWAY Consumer groups scale out and load-balance processing; retention is controlled by time/size policy; Kafka Streams is the stream-processing tool covered in this module; events, not direct calls, are the decoupling contract.

"It's about events, not calls."

MODULE 30 COMPLETE

Event-Driven Architecture and Apache Kafka

You can now design event-driven systems with Apache Kafka -- topics, partitions, producers, consumers, and consumer groups -- to build scalable, real-time, resilient enterprise architectures.